

融合算子多核 Tiling 策略指南

AscendC KernelLaunch · Atlas A2 · 以 int8 MatMul 实验为据

ascendc 工程

2026-06-19

摘要

本文档从 `ascendc-tests/int8-matmul-cube-128x512x512/` 的多核 CPU 实验归纳 **Host 侧 MatMul tiling** 与 **Launch** 的闭合条件, 并扩展到 **Cube + Vector 融合算子** (如 `Matmul-LeakyRelu`) 的配参思路。实验平台为 CANN 9.0.0、Atlas A2(910B4); 讨论纪要见 `qa/2026-06/2026-06-19-exp-int8-多核 tiling 实验.md`。

目录

1 适用范围与术语	2
1.1 适用范围	2
1.2 两层分块	2
1.3 Launch 与 <code>usedCoreNum</code> (A2)	2
2 Tiling 闭合条件 (用户定稿)	2
2.1 硬件与 base	2
2.2 宏块与微块	2
2.3 多核网格 (核心等式)	3
2.4 配参流程	3
3 实验用例: $128 \times 512 \times 512$ int8 MatMul	3
3.1 符号	3
3.2 已验证配置摘要	3
3.3 16 AIC 下三种等价 <code>SetSingleShape</code>	3
3.4 反例: 未闭合的 <code>SetSingleShape</code>	4
3.5 无 <code>SetSingleShape</code> 时的并行上界	4
4 融合算子扩展 (Matmul + Vector)	4
4.1 与纯 MatMul 的相同点	4
4.2 融合段额外注意	4
4.3 推荐 Host 模板 (伪代码)	4
5 故障诊断	5

1 适用范围与术语

1.1 适用范围

- **范式**: KernelLaunch; 算子在 *_custom.cpp, main.cpp 为应用壳。
- **硬件**: Atlas A2 (AIC:AIV = 1:2); 不覆盖 Ascend 310P 等特例。
- **矩阵乘段**: MultiCoreMatmulTiling + 高阶 Matmul<> (纯 Cube 或融合 Vector 段)。
- **dtype 示例**: $\text{int8} \times \text{int8} \rightarrow \text{int32}$ (实验); fp16 等需替换最小 tile 与 base 倍数。

1.2 两层分块

层级	Host 接口	决定内容
宏分片	SetDim、SetSingleShape、GetTiling	每个 blockIdx 负责 C 的子块
微分片	SetFixSplit(baseM, baseN, baseK)	每个 AIC 内部 L0/Cube 迭代粒度

常见误区: 认为只改 blockDim 与 SetDim 即可扩核。宏分块数由 GetTiling(或 SetSingleShape 闭合公式) 决定, 与 launch 数必须一致。

1.3 Launch 与 usedCoreNum (A2)

- main.cpp 中 blockDim: 本次 launch 的 **AI Core (AIC) 个数**。
- SetDim(usedCoreNum): 参与 MatMul 的 **AIV 规模**; 本工程取 $\text{usedCoreNum} = \text{blockDim} \times 2$ 。
- 内核用 GetBlockIdx() ($0 \dots \text{blockDim}-1$) 配合 CalcGMOffset 计算 GM 偏移。

2 Tiling 闭合条件 (用户定稿)

以下在 step/depth 不手改、baseK = -1 (K 向微块由 API 选择)、SingleCoreK = K (宏上 K 不切, 利于搬运与融合段衔接) 时成立。

2.1 硬件与 base

1. 实验设备 int8 入 / int32 出时, Cube 最小 tile 约为 $16 \times 32 \times 16$ ($M \times N \times K$)。
2. 故 baseM 为 **16 的倍数**, baseN 为 **32 的倍数**。
3. SetFixSplit(baseM, baseN, -1); baseK=-1 不表示 K 向宏切分。

2.2 宏块与微块

4. $\text{SingleCoreM} \bmod \text{baseM} = 0$, $\text{SingleCoreN} \bmod \text{baseN} = 0$ 。
5. L0A/L0B 须能容纳当前 base 与 SingleCore 下的搬入策略 (失败时常表现为 GetTiling 返回 -1 或 KFC 初始化错误)。

2.3 多核网格（核心等式）

$$\frac{M}{\text{SingleCoreM}} \times \frac{N}{\text{SingleCoreN}} = \text{usedCoreNum} \quad (1)$$

要求 M, N 与 $\text{SingleCoreM}, \text{SingleCoreN}$ 整除。等价形式： $\text{SingleCoreM} \times \text{SingleCoreN} \times \text{usedCoreNum} = M \times N$ 。 K 不参与式 (1)。

2.4 配参流程

1. 选定 usedCoreNum (= 宏分块总数 = $m_{\text{blk}} \times n_{\text{blk}}$)。
2. 将 usedCoreNum 因子分解，赋值 $m_{\text{blk}} = M / \text{SingleCoreM}$, $n_{\text{blk}} = N / \text{SingleCoreN}$ 。
3. 计算 $\text{SingleCoreM} = M / m_{\text{blk}}$, $\text{SingleCoreN} = N / n_{\text{blk}}$, $\text{SingleCoreK} = K$ 。
4. 校验对 base 的倍数关系与 L0。
5. `Host:SetSingleShape(SingleCoreM, SingleCoreN, K)` (推荐显式闭合), `SetDim(usedCoreNum)`, `SetFixSplit`。
6. `Launch: blockDim = usedCoreNum/2` (A2)。
7. `GetTiling` 后建议断言 $\text{res} \neq -1$, 并核对 `TCubeTiling` 中 $\text{singleCoreM}/N$ 与预期一致。

3 实验用例： $128 \times 512 \times 512$ int8 MatMul

3.1 符号

$A \in \mathbb{Z}^{128 \times 512}$, $B \in \mathbb{Z}^{512 \times 512}$, $C \in \mathbb{Z}^{128 \times 512}$, $\text{int8} \times \text{int8} \rightarrow \text{int32}$ 。

3.2 已验证配置摘要

AIC 数	usedCoreNum	SetFixSplit	SetSingleShape	结果
1	2	(16,32,*)	无	通过
4	8	(16,32,-1)	无	通过
8	16	(32,32,-1)	无	失败 (GetTiling / SIGFPE)
8	16	(16,32,-1)	无	通过 (自动约 8 路 N 切)
16	32	(16,32,-1)	无	失败 (launch 16, tiling 约 8 块)
16	32	(16,32,-1)	(32,64,K) 等三种	通过

3.3 16 AIC 下三种等价 SetSingleShape

$\text{usedCoreNum} = 32$, $\text{blockDim} = 16$, 均满足式 (1):

- $(32, 64, K)$: $4 \times 8 = 32$

- $(64, 32, K): 2 \times 16 = 32$
- $(16, 128, K): 8 \times 4 = 32$

差异在 M/N 切分几何与缓存局部性，非数学结果。

3.4 反例：未闭合的 SetSingleShape

$(128, 32, K): 1 \times 16 = 16 \neq 32 \Rightarrow \text{blockIdx} \geq 8$ 时 `tailN` 为负，`CheckTails` 断言失败。

3.5 无 SetSingleShape 时的并行上界

对 $N = 512$ 、 $\text{baseN} = 32$ ， N 向理论最大块数 $\lfloor N/\text{baseN} \rfloor = 16$ ；但 `GetTiling` 在本实验中自动常停在约 8 路 ($\text{singleCoreN} = 64$)。欲稳定达到 16 路，需按 §2 显式 `SetSingleShape` 闭合。

4 融合算子扩展 (Matmul + Vector)

4.1 与纯 MatMul 的相同点

- `MatMul` 段仍遵守 §2 全部条件；`REGIST_MATMUL_OBJ`、`CalcGMOffset`、`SetTail`（或 device 侧 `SetSingleShape`）与 `TCubeTiling` 一致。
- A2 上 `Cube:Vector = 1:2`；`baseM:baseN` 宜为 1:2（如 16:32），与 `int8` 最小 tile 一致。

4.2 融合段额外注意

- **VECIN/VEECOUT** 队列深度与 `stepM/stepN`：实验约定 **不手改**除非整体验证；改 `step` 会牵动 `Cube-Vector` 流水线。
- `Vector` 段按 $\text{singleCoreM} \times \text{singleCoreN}$ 子块消费 `Cube` 输出；宏分片错误时 `Cube` 与 `Vector` 同步错位。
- 多 AIC 融合算子在 CPU 孪生上可能遇跨核 `Matmul` 同步问题；优先保证 §2 闭合，再查融合 API；避免未闭合 tiling 时用多次 `launch` 掩盖根因（除非明确为 CPU 绕行）。

4.3 推荐 Host 模板（伪代码）

```
// 1. 定 usedCoreNum, 因子分解, 算 SingleCoreM/N
tilingApi.SetDim(usedCoreNum);
tilingApi.SetOrgShape(M, N, K);
tilingApi.SetShape(M, N, K);
tilingApi.SetSingleShape(singleCoreM, singleCoreN, K);
tilingApi.SetFixSplit(baseM, baseN, -1);
tilingApi.SetBufferSpace(-1, -1, -1);
if (tilingApi.GetTiling(tilingData) == -1) { /* abort */ }
// 2. launch
blockDim = usedCoreNum / 2; // A2
```

5 故障诊断

现象	优先检查
Ceiling SIGFPE	GetTiling 是否 -1; singleCoreM 是否为 0
tailN 负 / CheckTails	式 (1); blockDim 是否大于宏块数
totalIter_=0	该 blockIdx 无 N 向合法子块
部分 Block 成功	launch 块数 > Ceiling(M,sM)*Ceiling(N,sN)
改 base 后高核数才过	微块约束导致 GetTiling 在高并行度下无解

6 参考路径

- 实验目录: ascendc-tests/int8-matmul-cube-128x512x512/
- 讨论纪要: qa/2026-06/2026-06-19-exp-int8-matmul 多核 tiling 实验.md
- 模板: MatmulInvocationNeo、MatmulLeakyReluInvocation
- 平台纪要 §7.5: qa/2026-06/2026-06-09-AscendC 平台与 CANN 文档索引.md